Izmir Institute of Technology
Department of Computer Engineering

CENG 467 – Natural Language Understanding and Generation Take-Home Midterm Examination

Student Name: Zehra Eda Cesur
Student ID: 310201089
Instructor: Prof. Dr. Aytuğ Onan
Deadline: 30 Nisan 2026, 23:59

GitHub Repository:
https://github.com/zedaeda/CENG467-takeHomeMidterm

Spring 2026

Question 1. Representation Learning in Text Classification

1.1 Dataset Description

For this question, the IMDb Large Movie Review Dataset [11] was chosen for binary sentiment classification. The dataset contains movie reviews from the IMDb platform, each labeled as *positive* or *negative* based on the user's star rating.

Since the experiments were run on Google Colab's free tier with limited GPU resources, a subset of the full dataset was used: 5,000 samples for training, 1,000 for validation, and 2,000 for testing. As shown in Table 1, the class distribution is nearly balanced across all splits, which means the results are not affected by class imbalance.

Table 1: IMDb Dataset Statistics (Subset Used in Experiments)

Split	Samples	Positive	Negative
Train	5,000	2,506	2,494
Validation	1,000	488	512
Test	2,000	1,001	999

1.2 Preprocessing Pipeline and Tokenization

A common preprocessing pipeline was applied to all models where possible. Raw texts were lowercased and non-alphabetic characters were removed using regular expressions. Sequences longer than 256 tokens were truncated.

Two different tokenization strategies were compared:

Strategy A — Word-level tokenization (TF-IDF and BiLSTM): A simple whitespace and punctuation-based tokenizer was used, keeping only tokens with at least two characters. This approach is fast and easy to understand, but it cannot handle rare or unseen words well.

Strategy B — WordPiece tokenization (DistilBERT): The pre-trained `distilbert-base-uncased` tokenizer splits unknown words into smaller known subword units (e.g., *"playing"* → *"play"* + *"##ing"*). This eliminates out-of-vocabulary problems and preserves more morphological information. Sequences were padded or truncated to 128 tokens.

The main difference between these two strategies is that WordPiece gives richer representations for rare words, while word-level tokenization is much faster and simpler.

1.3 Model Implementations

1.3.1 TF-IDF + Logistic Regression (Sparse Baseline)

Two TF-IDF configurations were tested: unigram only $(1, 1)$ with 20,000 features, and unigram+bigram $(1, 2)$ with 30,000 features, both using sublinear TF scaling. A Logistic Regression classifier with C=1.0 was trained on these sparse vectors. The bigram configuration performed better on validation (Accuracy: 0.860 vs. 0.847) and was selected for [1]

[1] Code and report writing assistance was provided by Claude [1] and Gemini [3].

the final evaluation.

1.3.2 BiLSTM (Dense Neural Baseline)

A two-layer Bidirectional LSTM was implemented from scratch in PyTorch. Word embeddings of dimension 128 were randomly initialized and trained together with the model. The LSTM had a hidden size of 256 per direction, with 0.3 dropout. Training ran for 5 epochs using Adam (lr $= 10^{-3}$), gradient clipping at 1.0, and a batch size of 64.

1.3.3 DistilBERT (Contextual Transformer)

The `distilbert-base-uncased` checkpoint [4] was fine-tuned for sentiment classification by adding a classification head on top of the pre-trained encoder. Fine-tuning ran for 3 epochs with AdamW (lr $= 2 \times 10^{-5}$, weight decay $= 0.01$), a linear warmup over 10% of steps, and a batch size of 32.

1.4 Results

The test set results are shown in Table 2, and a visual comparison is given in Figure 1.

Table 2: Text Classification Results (Test Set)

Model	Accuracy (%)	Macro-F1 (%)	Training Time
TF-IDF + LR	87.25	87.25	∼2 min
BiLSTM	68.95	68.94	∼10 min
DistilBERT	86.60	86.60	∼25 min

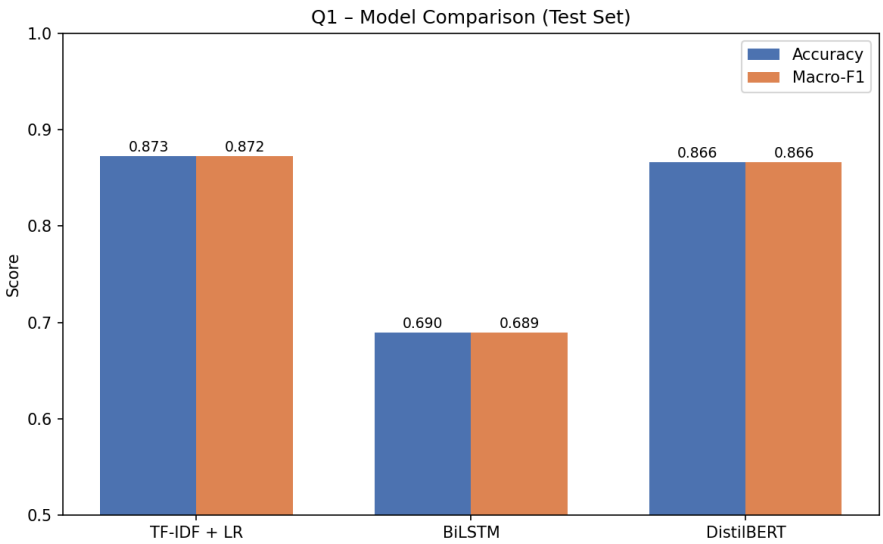


Figure 1: Accuracy and Macro-F1 comparison across the three models on the test set.

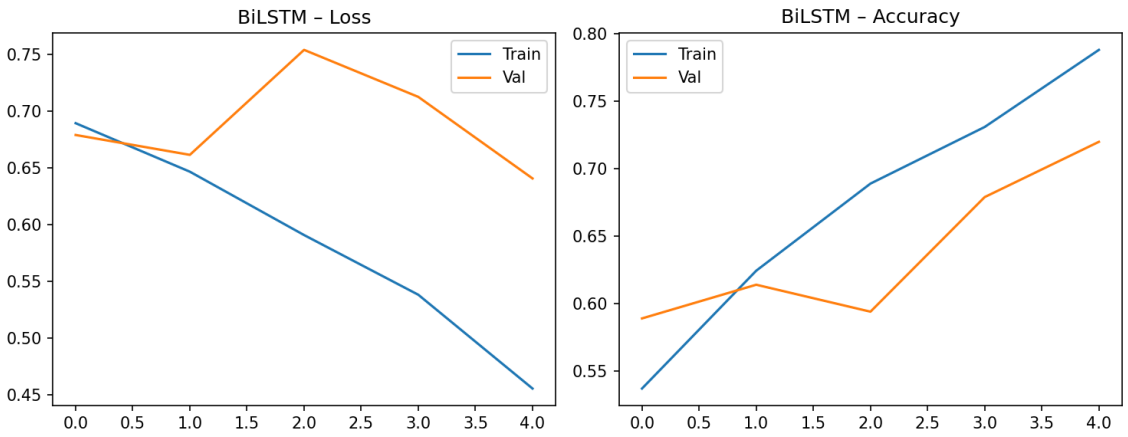


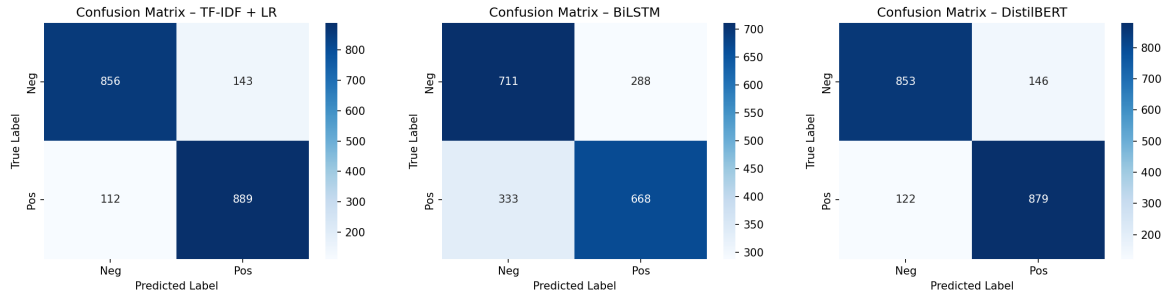
Figure 2: BiLSTM training and validation loss/accuracy over 5 epochs. The gap between train and validation loss after epoch 2 is a sign of overfitting.

1.5 Error Analysis

Table 3 shows five misclassified examples, and Figure 3 shows the confusion matrices for all three models.

Table 3: Representative Misclassified Examples

Text (truncated)	**True**	**Pred.**	**Error Type**
"Unfortunately, my old Blockbuster thought it was a good idea to have the sequel..."	Pos	Neg	Context loss
"I must assume the role of the turd in the punchbowl...encomia of praise..."	Neg	Pos	Irony / sarcasm
"real evidence of what a misguided and unchecked government can do..."	Pos	Neg	Domain shift
"Lucille Ball was a mighty power in television...but she still made an occasional film..."	Neg	Pos	Misleading opening
"I was Stan in the movie. Stan was the friend that...got his arm smashed..."	Neg	Pos	Ambiguous narrative



(a) TF-IDF + LR (b) BiLSTM (c) DistilBERT

Figure 3: Confusion matrices for all three models on the test set.

Looking at the errors, five common patterns stood out:

(1) Context loss: TF-IDF and BiLSTM struggled with reviews where the overall sentiment depends on the full narrative rather than individual words. A review starting with "unfortunately" about a missing film was labeled negative, even though the reviewer actually praised both films.

(2) Irony and sarcasm: Phrases like "encomia of praise" were used ironically, but all models treated them as positive signals. This kind of pragmatic reasoning is very hard for current models.

(3) Domain shift: Reviews about politically-charged topics used negative-sounding words like "misguided government" to describe film content, not the film quality itself. All three models were confused by this.

(4) Misleading opening sentences: Some negative reviews began with long positive descriptions of an actor or director, and only turned critical later. Models that focus heavily on early tokens were tricked by this pattern.

(5) Ambiguous narratives: Reviews written from a personal perspective (e.g., someone who acted in the film) did not have clear positive or negative signals, making correct classification difficult even for humans.

1.6 Discussion

The results were quite interesting and somewhat unexpected. The simplest model, TF-IDF + LR, actually performed the best (87.25%), while the more complex BiLSTM came in last (68.95%).

Why did TF-IDF win? With only 5,000 training examples, TF-IDF has a natural advantage: it does not need to learn anything from scratch. It simply counts word frequencies, which works surprisingly well for sentiment analysis where words like "terrible" or "amazing" are strong indicators. The BiLSTM, on the other hand, had to learn word meanings from scratch with very limited data, which led to clear overfitting.

Why did BiLSTM underperform? The training loss kept going down while the validation loss went up after epoch 2. This is a textbook case of overfitting. With the full 25,000-sample dataset, BiLSTM would likely perform much better.

What about DistilBERT? DistilBERT came very close to TF-IDF (86.60%) despite being fine-tuned on the same small subset. This shows the power of transfer learning — the model already knows a lot about language from its pre-training, so it needs less task-specific data. Its confusion matrix also shows the fewest total errors (268 vs. 621 for BiLSTM), meaning its mistakes are fewer even if its accuracy is similar to TF-IDF.

Regarding the effect of preprocessing decisions, stopword removal was applied during TF-IDF vectorization but not for BiLSTM or BERT, as neural models can learn to ignore uninformative tokens. Truncation at 256 tokens for BiLSTM and 128 for DistilBERT had minimal impact given that average review length was well within these limits.

In summary, for small datasets, simple methods can be surprisingly effective. But as data grows, contextual models like BERT are expected to pull ahead significantly [4].

Question 2. Named Entity Recognition

2.1 Dataset Description and BIO Tagging

The CoNLL-2003 dataset [14] was used for this task. It is one of the most widely-used benchmarks for Named Entity Recognition (NER). The dataset labels each token with one of four entity types: Person (**PER**), Organization (**ORG**), Location (**LOC**), and Miscellaneous (**MISC**).

The annotations follow the BIO tagging scheme (Begin, Inside, Outside), which in theory gives 9 unique tags: `O`, `B-PER`, `I-PER`, `B-ORG`, `I-ORG`, `B-LOC`, `I-LOC`, `B-MISC`, and `I-MISC`. In our downloaded version of the dataset, only `I-` prefixes were present (no `B-` prefixes), so 8 unique tags were used in practice.

For the BERT model, subword tokenization creates an alignment problem: a single word like "Washington" might be split into multiple tokens. To handle this, only the first subtoken of each word was assigned the original NER label. The remaining subtokens were assigned a dummy label (`-100`) that is ignored during loss computation.

Table 4: CoNLL-2003 Dataset Statistics

Split	Sentences	Tokens	Entities
Train	14,041	203,621	23,499
Validation	3,250	51,362	5,942
Test	3,453	46,435	5,648

2.2 Model Implementations

2.2.1 BiLSTM-CRF (Hybrid Architecture)

The first model combines a Bidirectional LSTM with a Conditional Random Field (CRF) layer [8, 9]. The BiLSTM reads each token in both directions and produces context-aware representations, while the CRF layer uses transition probabilities between tags to prevent invalid label sequences (e.g., going directly from `O` to `I-PER` is penalized).

Word embeddings of dimension 100 were randomly initialized and trained from scratch. The BiLSTM had a hidden dimension of 128. The model has approximately 2M parameters

and was trained for 10 epochs with Adam (lr $= 10^{-3}$).

2.2.2 BERT for Token Classification

The second model uses `distilbert-base-uncased` [4] with a linear token classification head on top. It was fine-tuned for 3 epochs with AdamW (lr $= 5 \times 10^{-5}$) and a linear learning rate scheduler with warmup. The model has approximately 66.3M parameters.

2.3 Results

Table 5: NER Results per Entity Type (Test Set)

Model / Entity	Precision	Recall	F1	Support
BiLSTM-CRF				
PER	0.0212	0.0155	0.0179	1617
ORG	0.1062	0.0566	0.0738	1661
LOC	0.1248	0.1145	0.1194	1668
MISC	0.0773	0.0627	0.0692	702
Overall	**0.0851**	**0.0627**	**0.0722**	**5648**
BERT-NER				
PER	0.9647	0.9635	0.9641	1616
ORG	0.8810	0.8736	0.8773	1661
LOC	0.9047	0.9286	0.9165	1667
MISC	0.7590	0.8077	0.7826	702
Overall	**0.8958**	**0.9074**	**0.9015**	**5646**

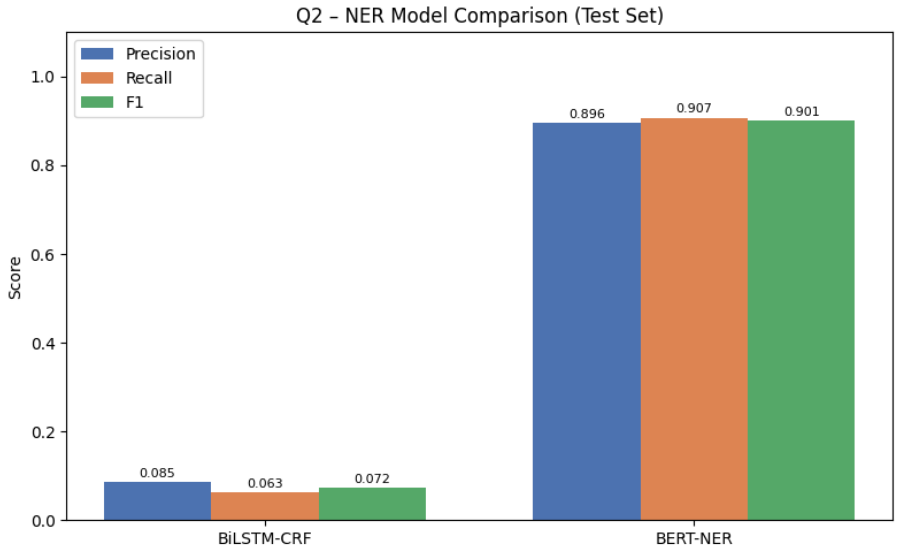


Figure 4: Precision, Recall, and F1 comparison between BiLSTM-CRF and BERT-NER on the CoNLL-2003 test set.

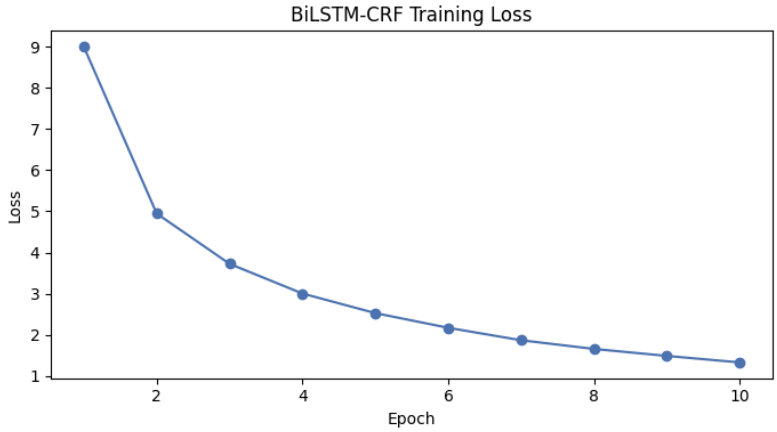


Figure 5: BiLSTM-CRF training loss over 10 epochs.

2.4 Error Analysis and Discussion

The gap between the two models was very large: BERT-NER reached an overall F1 of 0.90, while BiLSTM-CRF only reached 0.07. This was the most striking result of the entire midterm for me, so I want to explain it carefully.

The BiLSTM-CRF's poor performance comes down to one main reason: its embeddings were initialized randomly and trained from scratch on the CoNLL data alone. The standard practice for BiLSTM-CRF is to initialize embeddings with pre-trained GloVe vectors, which already encode word meanings [9]. Without this, the model had no prior knowledge of what words mean, so it mostly predicted `O` for everything and occasionally guessed wrong entity types. For example, it predicted commas as `I-LOC` and missed obvious entities like "JAPAN".

BERT's errors were much more interesting. For instance, it labeled "CHINA" as `I-LOC` in the sentence "CHINA IN SURPRISE DEFEAT", when the dataset labeled it as `I-PER` (referring to a sports team). BERT was not wrong in a linguistic sense — it simply applied its geographic prior instead of understanding the sports context. It also occasionally missed `MISC` entities like "1995" in "1995 World Cup".

These results confirm that pre-trained contextual embeddings are essential for NER. The CRF layer helps enforce valid tag sequences, but it cannot make up for the lack of good token representations.

Question 3. Text Summarization

3.1 Dataset Description

The CNN/DailyMail dataset [6] was used for this task. It pairs news articles with human-written bullet-point summaries, making it a standard benchmark for summarization. Due to computational constraints, only 500 randomly sampled articles from the test split were used for evaluation.

Table 6: CNN/DailyMail Subset Statistics

Property	**Value**
Subset size (test)	500 articles
Avg. article length	687 words
Avg. reference summary length	56 words

3.2 Model Implementations

3.2.1 TextRank (Extractive)

TextRank [13] is a graph-based algorithm adapted from PageRank. Each sentence is a node, and edges are weighted by the overlap between sentences (shared non-stopword tokens, normalized by sentence length). A damping factor of 0.85 was used, with convergence tolerance 10^{-6} and a maximum of 100 iterations. The top-3 ranked sentences, in their original order, form the summary. TextRank runs entirely on CPU and needs no training data.

3.2.2 BART (Abstractive)

BART [10] is a sequence-to-sequence transformer pre-trained with a denoising objective. The `facebook/bart-large-cnn` checkpoint was used directly without additional fine-tuning, since it was already fine-tuned on CNN/DailyMail. Articles were truncated to 3,000 characters before being fed to the model. Summaries were generated with greedy decoding, a maximum length of 130 tokens, and a minimum of 30 tokens.

3.3 Quantitative Results

Table 7: Summarization Evaluation Results (500-sample Test Subset)

Metric	**TextRank**	**BART**
ROUGE-1	0.3559	**0.4380**
ROUGE-2	0.1392	**0.2121**
ROUGE-L	0.2279	**0.3082**
BLEU	0.0923	**0.1810**
METEOR	0.3419	**0.3817**
BERTScore	0.8653	**0.8819**

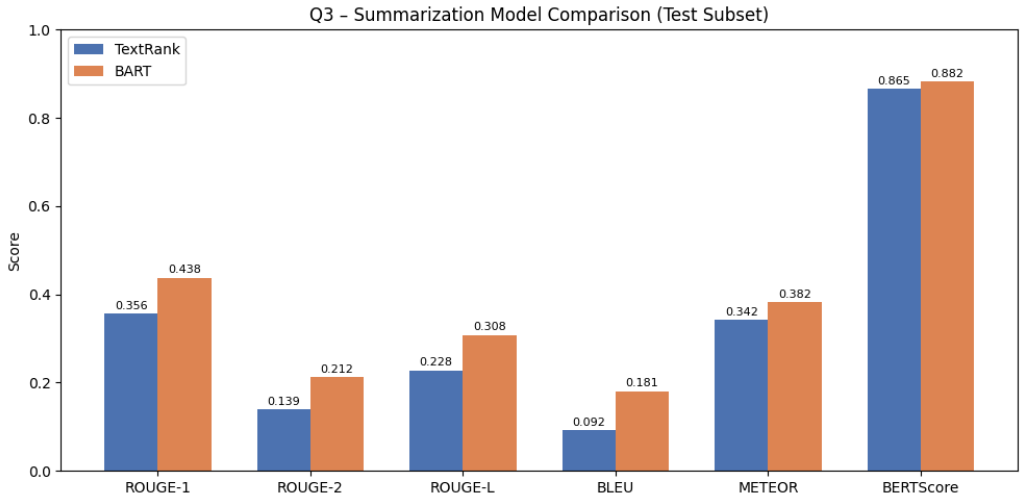


Figure 6: Metric-by-metric comparison of TextRank and BART on the CNN/DailyMail test subset.

3.4 Qualitative Examples

Example 1 — Medical Marijuana Article
Reference: CNN's Dr. Sanjay Gupta argues for legalizing medical marijuana, acknowledging his own inaction on the issue.
TextRank: Picked three sentences from the article body, capturing the political context but missing the author's personal stance and the key statistic (53% majority support).
BART: Correctly named the author, included the 53% figure, and summarized the trend concisely.
Analysis: BART was better at pulling together scattered facts from different parts of the article, which TextRank simply cannot do.

Example 2 — Child Gang Life on Twitter
Reference: Focused on the Twitter account, the gun video, and public backlash.
TextRank: Covered the visuals and backlash but copied offensive tweet content verbatim since it cannot paraphrase.
BART: Produced a more fluent summary but also included some offensive content from the source text.
Analysis: Neither model can filter sensitive content. TextRank is completely stuck with the source sentences, while BART at least partially reorganizes the information.

Example 3 — Chris Christie Town Hall
Reference: Detailed the teacher's question, Christie's response, and the union's reaction.
TextRank: Selected three good sentences covering the key actors and the ExxonMobil settlement.
BART: Generated a fluent three-sentence summary that also included the $225 million figure and a direct quote from the union spokesperson.
Analysis: BART integrated specific numbers and quotes from different parts of the article into a coherent narrative.

3.5 Discussion

BART outperformed TextRank on all six metrics, but the BERTScore gap was surprisingly small (0.865 vs. 0.882). This means that while TextRank's outputs do not match the reference wording closely (hence lower ROUGE and BLEU), they still carry similar semantic content.

ROUGE and BLEU measure n-gram overlap. TextRank scores lower because reference summaries are written in a compressed, paraphrased style, while TextRank copies full sentences. BART's generated text is closer in style to the references.

METEOR uses synonym matching, which is why both models scored more similarly here than on ROUGE.

BERTScore is based on semantic similarity via contextual embeddings. The narrow gap here suggests that extracted sentences — even if not perfectly worded — convey the right meaning.

Key trade-offs between the two approaches:

- *Computational cost:* TextRank is unsupervised and runs on CPU in minutes. BART needs GPU and a large model download.
- *Faithfulness:* TextRank is always faithful to the source since it only extracts original sentences. BART can occasionally hallucinate.
- *Readability:* BART produces more fluent summaries. TextRank can feel disjointed since its sentences come from different parts of the article.
- *Flexibility:* BART can control output length and style. TextRank is limited to picking a fixed number of source sentences.

Question 4. Machine Translation

4.1 Dataset Description and Preprocessing

The Multi30k dataset [5] was used for English-to-German (EN→DE) translation. It contains 29,000 training, 1,014 validation, and 1,000 test sentence pairs describing images. Average sentence length is 11.9 words in English and 11.1 words in German. Despite being small, it is a challenging benchmark due to German's complex morphology.

Preprocessing included lowercasing, punctuation separation, and filtering of non-alphabetic characters. Special tokens `<sos>`, `<eos>`, `<pad>`, and `<unk>` were added. Words appearing fewer than twice were discarded, giving a source vocabulary of 6,290 and a target vocabulary of 8,431 tokens. Sequences were truncated at 50 tokens.

4.2 Model Implementations

4.2.1 Seq2Seq with Bahdanau Attention

A GRU-based encoder-decoder model with Bahdanau attention [2] was implemented from scratch. The bidirectional encoder maps source tokens to 512-dimensional hidden states. At each decoding step, the attention mechanism computes a weighted sum of all encoder outputs (the context vector), which is concatenated with the decoder input. The model

has 25.3M parameters and was trained for 10 epochs using Adam (lr = 10^{-3}), gradient clipping at 1.0, batch size 128, and teacher forcing ratio 0.5.

4.2.2 Helsinki-NLP Transformer (Pre-trained)

The `Helsinki-NLP/opus-mt-en-de` MarianMT model [15] was used directly for inference without any fine-tuning. It has 133.9M parameters and was pre-trained on the large OPUS multilingual corpus. Translation used beam search with 4 beams.

4.3 Quantitative Results

Table 8: Machine Translation Results (Multi30k Test Set)

Metric	Seq2Seq+Attn	Transformer
BLEU	4.94	**36.24**
METEOR	56.97	**66.68**
ChrF	43.04	**64.25**
BERTScore	76.45	**89.66**

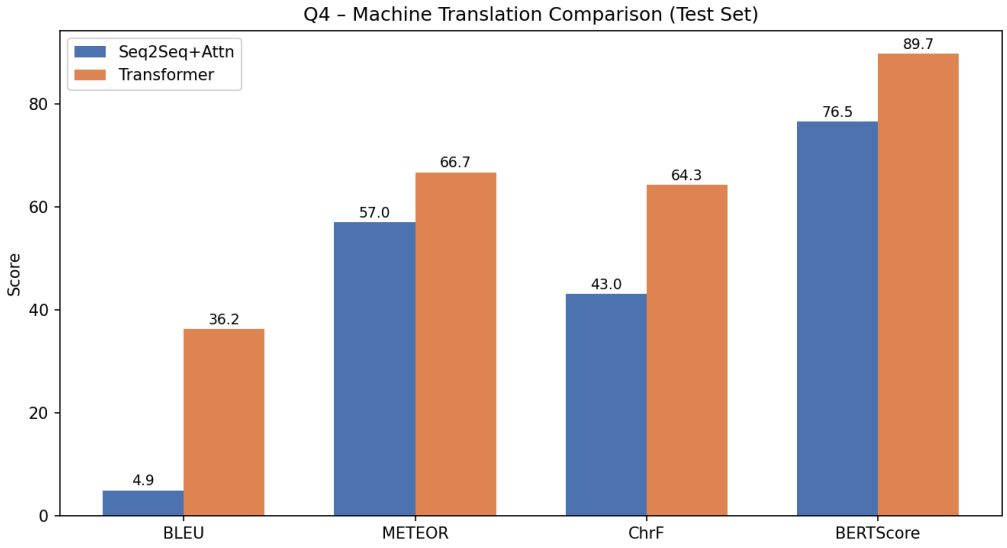


Figure 7: Metric comparison between Seq2Seq+Attention and Helsinki Transformer on the Multi30k test set.

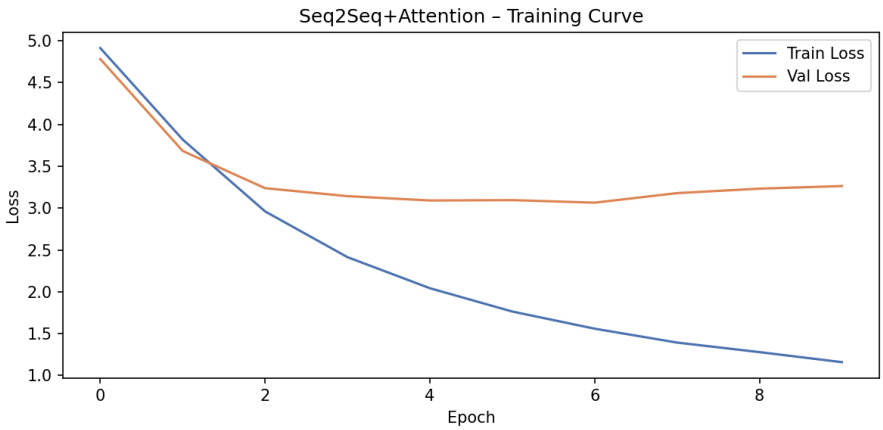


Figure 8: Seq2Seq+Attention training and validation loss over 10 epochs. Overfitting begins from epoch 4 onward.

4.4 Qualitative Example

Table 9: Translation Examples from the Multi30k Test Set

	Example 1
Source (EN)	A man in an orange hat starring at something.
Reference (DE)	Ein Mann mit einem orangefarbenen Hut, der etwas anstarrt.
Seq2Seq	ein mann mit einem orangefarbenen hut starrt etwas etwas an .
Transformer	Ein Mann in einem orangenen Hut, der mit etwas zu tun hat.
	Example 2
Source (EN)	A Boston Terrier is running on lush green grass in front of a white fence.
Reference (DE)	Ein Boston Terrier läuft über saftig-grünes Gras vor einem weißen Zaun.
Seq2Seq	ein <unk> hirtenhund rennt auf grünes grünes gras vor einem weißen zaun .
Transformer	Ein Boston Terrier läuft auf üppig grünem Gras vor einem weißen Zaun.
	Example 3
Source (EN)	A girl in karate uniform breaking a stick with a front kick.
Reference (DE)	Ein Mädchen in einem Karateanzug bricht ein Brett mit einem Tritt.
Seq2Seq	ein mädchen in <unk> wird einen einem mit einem stock .
Transformer	Ein Mädchen in Karate-Uniform bricht einen Stock mit einem Front-Kick.

In Example 2, Seq2Seq could not translate "Boston Terrier" because it was outside its small vocabulary, so it produced `<unk>` and then guessed "Hirtenhund" (shepherd dog), which is completely wrong. In Example 3, the output "wird einen einem" is grammatically incoherent, showing that the model could not handle the morphological agreement rules of German. The Transformer handled both cases correctly.

4.5 Discussion

The Transformer outperformed Seq2Seq on all metrics by a large margin.

BLEU (4.94 vs. 36.24) is the harshest metric here because it requires exact word matches. Seq2Seq's many `<unk>` tokens and word repetitions (e.g., "grünes grünes") severely hurt its BLEU score.

METEOR (56.97 vs. 66.68) is more forgiving thanks to stemming and synonym matching. Seq2Seq still gets partial credit when it produces the right root word even if the inflection is wrong.

ChrF (43.04 vs. 64.25) works at the character level, which is useful for German because of its many inflected word forms. The Transformer scored much higher here as well.

BERTScore (76.45 vs. 89.66) shows that even the weak Seq2Seq model produces semantically relevant output most of the time. This suggests that the model learned some meaningful word associations, even if the word forms and order are often wrong.

The training curve (Figure 8) shows clear overfitting from epoch 4 onward. Training on only 29,000 sentence pairs is simply not enough to learn the complexity of German. The Transformer, pre-trained on millions of sentence pairs, had a huge head start.

Question 5. Language Modeling

5.1 Dataset Description

The WikiText-2 dataset [12] was used. It consists of Wikipedia articles and is a standard benchmark for language modeling. Compared to Penn Treebank, it has a larger vocabulary and longer documents.

Table 10: WikiText-2 Dataset Statistics

Split	Sentences	Tokens
Train	17,556	2,007,146
Validation	1,841	209,338
Test	2,183	235,845

A vocabulary of 10,000 tokens was built from the training set. Section headings were excluded. Words outside the vocabulary were mapped to `<unk>`.

5.2 Model Implementations

5.2.1 Bigram Language Model

A bigram model predicts each word given only the previous word: $P(w_i \mid w_{i-1})$. Laplace (add-one) smoothing ($\alpha = 1.0$) was applied to handle unseen word pairs. This model is count-based and requires no training.

5.2.2 Trigram Language Model

A trigram model conditions on the two previous words: $P(w_i \mid w_{i-2}, w_{i-1})$. The same Laplace smoothing was applied. While theoretically capturing more context, it is much more affected by data sparsity, as explained in the Discussion.

5.2.3 LSTM Language Model

A two-layer LSTM [7] was implemented. Token embeddings of dimension 512 were fed into two LSTM layers with hidden size 512. Weight tying was applied between the embedding matrix and the output projection layer. Dropout of 0.5 was used throughout. Training ran for 15 epochs using SGD (initial lr = 20.0), with learning rate halving from epoch 10 onward and gradient clipping at 0.25. BPTT window size was 35, batch size 64. The model has approximately 9.3M parameters.

5.3 Results

Table 11: Language Model Perplexity (lower is better)

Model	Val. Perplexity	Test Perplexity
Bigram	1,473.88	1,504.73
Trigram	5,287.55	5,341.04
LSTM	**70.52**	**66.77**

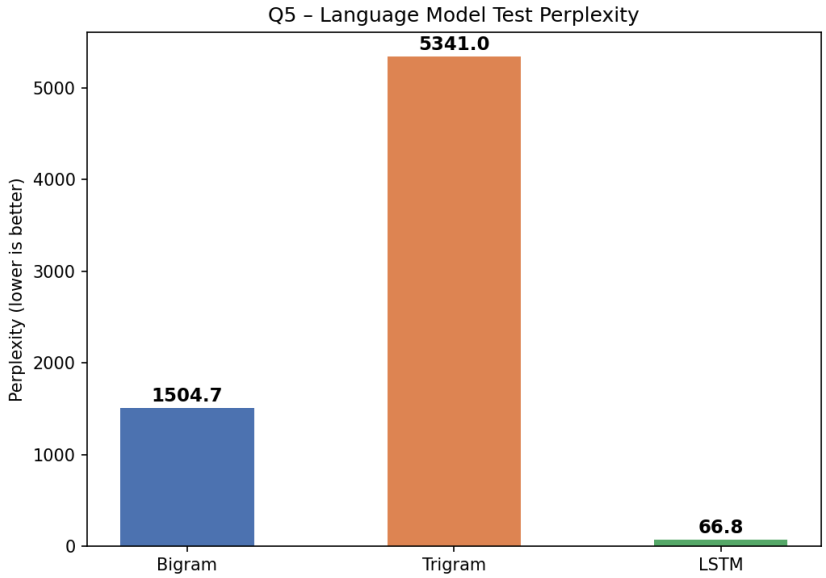


Figure 9: Test perplexity comparison across the three language models.

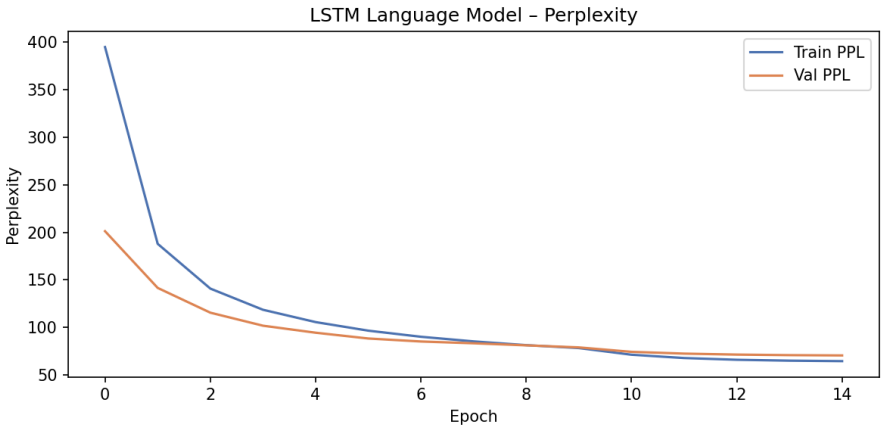


Figure 10: LSTM training and validation perplexity over 15 epochs. Learning rate decay from epoch 10 accelerates convergence.

5.4 Generated Text Samples

Table 12: Generated Text Samples (seed words in bold)

Model	Generated Text
Bigram	**the** canadian / soul of the local tradition by dave thomas 's misdirected martyrdom of the 1st , that " a
Bigram	**he** was able to mogadishu .
Trigram	**the** junta to become the show received a message back for more than one cult center , which travels to sarasaland
Trigram	**he** nails it " her two elder pitman children received their offerings in bodies of the 2004 mtv video music awards
LSTM	**the** <unk> <unk> and the <unk> of the <unk> .
LSTM	**he was** the only choice of the history of the city .

5.5 Discussion

Perplexity results. The LSTM dramatically outperformed both n-gram models, reaching a test perplexity of 66.77 compared to 1,504.73 for Bigram and 5,341.04 for Trigram.

The most surprising finding was that the Trigram performed *worse* than the Bigram, even though it uses more context. The reason is data sparsity combined with Laplace smoothing. With a 10,000-word vocabulary, there are $10{,}000^2 = 10^8$ possible trigram contexts — far more than actually appear in the training data. Laplace smoothing adds $\alpha \cdot V$ counts to the denominator of every context, which effectively pushes all probabilities toward uniform. Kneser-Ney smoothing would handle this much better.

Training dynamics. The LSTM's perplexity curve (Figure 10) drops quickly in the first five epochs and then continues to improve steadily after learning rate decay kicks in at epoch 10. Importantly, training and validation curves stay close to each other throughout — there is no sign of overfitting. This is thanks to the high dropout (0.5) and weight tying.

Generation quality. Bigram outputs are locally plausible (e.g., "he was able to...") but jump between unrelated topics from word to word. Trigram outputs are slightly more coherent in short spans but are still semantically disconnected overall. LSTM outputs suffer from many <unk> tokens because the 10,000-token vocabulary is much smaller than WikiText-2's full 33,000-token vocabulary. However, the "he was" sample produced a grammatically correct and meaningful sentence, showing the LSTM has learned real language structure.

Summary. N-gram models are fast and need no GPU, but they are severely limited by data sparsity and the Markov assumption. The LSTM overcomes both problems by learning dense representations and capturing long-range dependencies, though it requires more resources and training time.

References

[1] Anthropic. Claude (claude-sonnet-4-5). Large language model, https://claude.ai, 2026. Used for code assistance and report writing support.

[2] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. In *Proceedings of ICLR*, 2015.

[3] Google DeepMind. Gemini. Large language model, https://gemini.google.com, 2026. Used for code assistance and report writing support.

[4] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*, pages 4171–4186, 2019.

[5] Desmond Elliott, Stella Frank, Khalil Sima'an, and Lucia Specia. Multi30K: Multilingual english-german image descriptions. In *Proceedings of the 5th Workshop on Vision and Language*, pages 70–74, 2016.

[6] Karl Moritz Hermann, Tomáš Kočiský, Edward Grefenstette, Lasse Espeholt, Will Kay, Mustafa Suleyman, and Phil Blunsom. Teaching machines to read and comprehend. In *Advances in NeurIPS*, pages 1693–1701, 2015.

[7] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.

[8] John Lafferty, Andrew McCallum, and Fernando C. N. Pereira. Conditional random fields: Probabilistic models for segmenting and labeling sequence data. pages 282–289, 2001.

[9] Guillaume Lample, Miguel Ballesteros, Sandeep Subramanian, Kazuya Kawakami, and Chris Dyer. Neural architectures for named entity recognition. In *Proceedings of NAACL-HLT*, pages 260–270, 2016.

[10] Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Veselin Stoyanov, and Luke Zettlemoyer. BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In *Proceedings of ACL*, pages 7871–7880, 2020.

[11] Andrew L. Maas, Raymond E. Daly, Peter T. Pham, Dan Huang, Andrew Y. Ng, and Christopher Potts. Learning word vectors for sentiment analysis. pages 142–150, 2011.

[12] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. *arXiv preprint arXiv:1609.07843*, 2016.

[13] Rada Mihalcea and Paul Tarau. TextRank: Bringing order into text. In *Proceedings of EMNLP*, pages 404–411, 2004.

[14] Erik F. Tjong Kim Sang and Fien De Meulder. Introduction to the CoNLL-2003 shared task: Language-independent named entity recognition. In *Proceedings of CoNLL*, pages 142–147, 2003.

[15] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in NeurIPS*, pages 5998–6008, 2017.